

Computational Thinking and the Foundation of Artificial Intelligence

Introduction

Dr. Elena Fedorchenko, Associate Professor

CUMT

2026–2027



About me



Elena Fedorchenko (maiden name – Doynikova)

Ph.D., Associate Professor

Contacts: elena.doinikova86@gmail.com

Scientific career

from 2010: Senior Researcher, Laboratory of Computer Security Problems,
St. Petersburg Federal Research Center of the Russian Academy of
Sciences (SPC RAS);
Associate Professor, The Bonch-Bruевич Saint Petersburg State
University of Telecommunications (SPbSUT),
Associate Professor, The St. Petersburg State Electrotechnical
University "LETI"

before 2010: Development Engineer, QA Engineer, QA Team Lead

About me



Elena Fedorchenko (maiden name – Doynikova)

Ph.D., Associate Professor

Contacts: elena.doinikova86@gmail.com

Main research interests

Information security and privacy, risk management, security measuring, decision support; application of AI in information security and privacy; data analysis; ontology and knowledge graphs

Author more than 100 papers, 2 monographs and 4 patents.

Google Scholar



Scopus



ORCID



ResearchGate



Course structure 1/3

72 hours (20 lectures, 16 practices)

- Introduction
- Computational thinking and intelligent systems
- Knowledge representation and reasoning
- Search, uncertainty and intelligent decision-making
- Learning from data
- AI in engineering practice
- Conclusion

Course structure 2/3

- Introduction
- Computational thinking and intelligent systems
 - Computational thinking
 - Concepts, brief history of AI and building intelligent systems
 - Data, search and problem solving in intelligent engineering systems
- Knowledge representation and reasoning
 - Propositional logic for engineering decisions
 - First-order logic and knowledge representation
 - Rule-based expert systems
 - Frame-based expert systems, semantic networks and knowledge graphs
 - Ontologies and structured knowledge
- Search, uncertainty and intelligent decision-making
 - Information retrieval, semantic search and generative AI
 - Uncertainty and probabilistic reasoning
 - Fuzzy logic and fuzzy sets
 - Fuzzy inference and fuzzy expert systems
 - Optimization and evolutionary computation

Course structure 3/3

- Learning from data
 - Machine learning fundamentals
 - Machine learning workflow and model evaluation
 - Artificial Neural Networks
 - Computer vision for mining and safety
 - Time series, anomaly detection and predictive maintenance
- AI in engineering practice
 - Industrial and trustworthy AI
- Conclusion
 - Course review and test preparation

Sources

- *Michael Negnevitsky. Artificial Intelligence: A Guide to Intelligent Systems (3d Edition), 2011*
- *Course on Foundations of Artificial Intelligence by Dr. Ryan Urbanowicz, University of Pennsylvania (2020)*
(<https://www.youtube.com/watch?v=NVYSnFZwMKk&list=PLafPhSv10SDew4qhV9YeNFwf-1eUONHng>)
- Other

Requirements

Prerequisites:

- Higher math, basics
- Additionally: Python, basics

Assessment:

- Computational Thinking and the Foundation of Artificial Intelligence
 - Quizzes 20%
 - Exam (test) 70%
 - Work in class: presence, questions/answers 10%
- Computational Thinking and the Foundation of Artificial Intelligence: experiment
 - Implement 16 practical tasks 70%
 - Course work 30%

Course structure 2/3

- Introduction
- Computational thinking and intelligent systems
 - Computational thinking
 - Concepts, brief history of AI and building intelligent systems
 - Data, search and problem solving in intelligent engineering systems
- Knowledge representation and reasoning
 - Propositional logic for engineering decisions
 - First-order logic and knowledge representation
 - Rule-based expert systems
 - Frame-based expert systems, semantic networks and knowledge graphs
 - Ontologies and structured knowledge
- Search, uncertainty and intelligent decision-making
 - Information retrieval, semantic search and generative AI
 - Uncertainty and probabilistic reasoning
 - Fuzzy logic and fuzzy sets
 - Fuzzy inference and fuzzy expert systems
 - Optimization and evolutionary computation

- 1 Lecture 1: Computational thinking
- 2 Conclusion
- 3 Check your understanding

Computational thinking

- A systematic approach akin to the management of computer systems, essential for analytical reasoning across diverse fields.

Shenmeihao Tian, 2024

Computational thinking: components

Computational Thinking is derived from the methodologies employed by the computers to address problems, allowing for the seamless integration of its components into computing systems.

Components:

- Trouble decomposition,
- Abstraction,
- Pattern recognition,
- Algorithms, and
- Debugging.

Computational thinking: trouble decomposition

Trouble decomposition

- Breaking down complex problems into smaller, manageable tasks.

Shenmeihao Tian, 2024

Computational thinking: example

Case

- Methane safety monitoring in an underground mine.

Main problem

- How can we monitor methane concentration and detect potentially dangerous conditions in an underground mine?

Computational thinking: trouble decomposition example

Main problem

- How can we monitor methane concentration and detect potentially dangerous conditions in an underground mine?

Trouble decomposition

- Measure methane concentration,
- Collect data from sensors,
- Monitor changes over time,
- Compare values with safety criteria,
- Detect potentially dangerous conditions,
- Generate an alert, and
- Provide information to the operator.

Computational thinking: abstraction

Abstraction

- Explaining how to focus on essential details while ignoring irrelevant information.

Shenmeihao Tian, 2024

Computational thinking: example

Methane safety monitoring in an underground mine

- Measure methane concentration,
- Collect data from sensors,
- Monitor changes over time,
- Compare values with safety criteria,
- Detect potentially dangerous conditions,
- Generate an alert, and
- Provide information to the operator.

Computational thinking: abstraction example

Methane safety monitoring in an underground mine

- Measure methane concentration,
- Collect data from sensors,
- Monitor changes over time,
- Compare values with safety criteria,
- Detect potentially dangerous conditions,
- Generate an alert, and
- Provide information to the operator.

Relevant information

- methane concentration,
- sensor location,
- measurement time,
- airflow / ventilation status,
- safety threshold,
- sensor status / reliability,
- change of methane concentration over time.

Computational thinking: abstraction example

Relevant information

- methane concentration,
- sensor location,
- measurement time,
- airflow / ventilation status,
- safety threshold,
- sensor status / reliability,
- change of methane concentration over time.

Irrelevant information

- internal construction of the methane sensor,
- detailed design of tunnels and mine structure,
- internal structure of ventilation equipment,
- construction details of mining machines,
- details of the communication hardware.

Instead of modeling the entire mine, represent each measurement as:

Sensor ID -> Location -> Time -> CH_4 value -> Status.

Example:

S12 -> Tunnel 3 -> 10:32 -> 1.2% CH_4 -> Active.

Computational thinking: algorithms

Algorithms

- Describing step-by-step procedures for solving problems or performing tasks.

Shenmeihao Tian, 2024

Computational thinking: algorithms example

Methane safety monitoring in an underground mine

- Measure methane concentration,
- Collect data from sensors,
- Monitor changes over time,
- **Compare values with safety criteria,**
- Detect potentially dangerous conditions,
- Generate an alert, and
- Provide information to the operator.

Algorithm

- 1 Receive the current CH_4 measurement.
- 2 Check the applicable safety criterion.
- 3 Compare the measured value with the criterion.
- 4 Determine whether the value is within the acceptable range.
- 5 Pass the result to the hazard-detection stage.

Computational thinking: algorithms example

Methane safety monitoring in an underground mine

- Measure methane concentration,
- Collect data from sensors,
- Monitor changes over time,
- Compare values with safety criteria,
- **Detect potentially dangerous conditions,**
- Generate an alert, and
- Provide information to the operator.

Algorithm

- ① Receive CH_4 value.
- ② Receive location and time.
- ③ Check recent measurements.
- ④ Compare current conditions with safety criteria.
- ⑤ Classify the condition as normal or potentially hazardous.
- ⑥ Pass the result to the alert subsystem.

Computational thinking: pattern recognition

Pattern recognition

- Identifying similarities or recurring themes in data or problems.

Shenmeihao Tian, 2024

Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.5% -> 0.6% -> 0.8% -> 1.0%

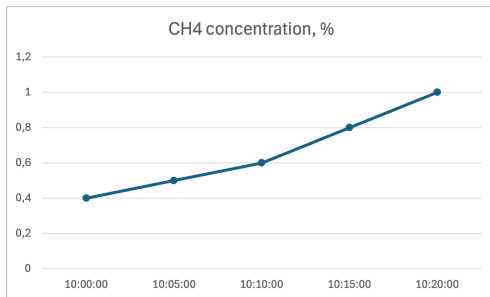
Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.5% -> 0.6% -> 0.8% -> 1.0%



Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.5% -> 1.1% -> 0.5% -> 0.4%

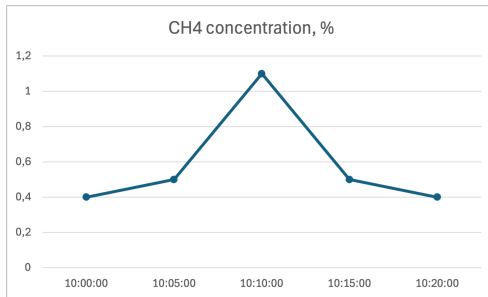
Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.5% -> 1.1% -> 0.5% -> 0.4%



Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.9% -> 0.4% -> 0.9% -> 0.4% -> 0.9%

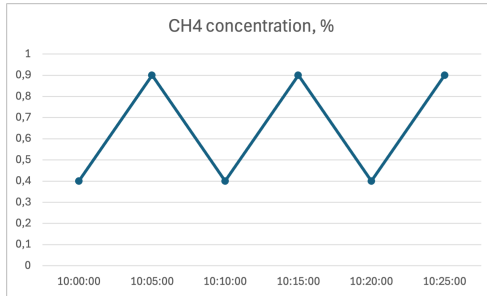
Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

The methane concentration over time

- 0.4% -> 0.9% -> 0.4% -> 0.9% -> 0.4% -> 0.9%



Computational thinking: pattern recognition example

Case

- Methane safety monitoring in an underground mine.

Single measurement defines the current state

$$CH_4 = 0.8\%$$

Series of measurements show the pattern over time

0.4% -> 0.5% -> 0.6% -> 0.8% -> 1.0%

Recognizing pattern helps us understand how a situation is developing, not only what is happening at one moment.

Computational thinking: debugging/troubleshooting

Debugging/troubleshooting

- Identifying and fixing errors or issues in algorithms or programs.

Shenmeihao Tian, 2024

Computational thinking: Debugging/troubleshooting example

Case

- Methane safety monitoring in an underground mine.
- The methane monitoring system suddenly generates an alarm, although the actual methane level is normal.

Computational thinking: Debugging/troubleshooting example

Case

- The methane monitoring system suddenly generates an alarm, although the actual methane level is normal.

Possible reasons

- the methane sensor is faulty,
- the sensor is incorrectly calibrated,
- data are transmitted incorrectly,
- the wrong safety threshold is used,
- the algorithm processes the value incorrectly,
- the sensor location is assigned incorrectly,
- old measurements are being used instead of current data.

Computational thinking: Debugging/troubleshooting example

Case

- The methane monitoring system suddenly generates an alarm, although the actual methane level is normal.

Troubleshooting process

- 1 Identify the unexpected behavior: *The system generates a false alarm.*
- 2 Check possible sources of the problem: *Sensor -> data transmission -> threshold -> algorithm.*
- 3 Test each hypothesis: *Compare the sensor reading with another sensor or reference measurement.*
- 4 Find the cause. *For example: incorrect sensor calibration.*
- 5 Correct the problem and test again

Computational thinking: Debugging/troubleshooting example

Troubleshooting process

- ① Identify the unexpected behavior: *Alarm is triggered at $CH_4 = 0.4\%$.*
- ② Check possible sources of the problem: *Sensor -> data transmission -> threshold -> algorithm.*
- ③ Test each hypothesis:
 - Hypothesis 1: sensor is wrong -> compare with another sensor.
 - Hypothesis 2: threshold is wrong -> inspect threshold setting in a system.
 - Hypothesis 3: algorithm is wrong -> test with known input values.
- ④ Find the cause. *For example: sensor is wrong.*
- ⑤ Correct the problem and test again

From computational thinking to artificial intelligence

Computational thinking

- Helps us formulate a problem clearly.

Artificial intelligence

- Provides methods that can help solve some of these problems automatically.

From computational thinking to artificial intelligence

Computational thinking

- decompose the monitoring problem,
- identify essential information,
- recognize patterns,
- define algorithms,
- troubleshoot errors.

Artificial intelligence

- detect unusual methane patterns,
- estimate risk from several sensor signals,
- classify conditions as normal / hazardous,
- support operator decisions,
- generate warnings or recommendations.

- ① Lecture 1: Computational thinking
- ② Conclusion
- ③ Check your understanding

Conclusion

When is a fixed algorithm enough, and when do we need AI?

- In case of simple rule, a fixed algorithm is enough:
 $CH_4 > threshold \rightarrow alarm.$
- If there are a lot of signals, complex dependencies, trends and uncertainty $\rightarrow$ AI.

- 1 Lecture 1: Computational thinking
- 2 Conclusion
- 3 Check your understanding

Task 1: Identify the computational thinking component

A methane monitoring system records the following CH_4 values:

- 0.4% -> 0.5% -> 0.6% -> 0.8% -> 1.0%.

What computational thinking component are we using when we notice that methane concentration is increasing?

Task 2: Abstraction

A monitoring system receives information about:

- methane concentration,
- sensor location,
- measurement time,
- sensor status,
- internal sensor electronics,
- detailed tunnel construction,
- equipment design.

What information should be included in a simplified model for methane monitoring?

Task 3: Decomposition

Problem: An underground mine needs a system to monitor the condition of a ventilation fan.

- Break this problem into 4-5 smaller tasks.

Task 3: Decomposition

Problem: An underground mine needs a system to monitor the condition of a ventilation fan.

- collect operating data,
- monitor vibration / temperature,
- compare measurements with normal conditions,
- detect abnormal behavior,
- generate a warning,
- provide information to the operator.

Task 4: Algorithms

Problem: An underground mine needs a system to monitor the condition of a ventilation fan.

- collect operating data,
- monitor vibration / temperature,
- compare measurements with normal conditions,
- detect abnormal behavior,
- generate a warning,
- provide information to the operator.

Choose one subproblem: Detect abnormal fan temperature.
Write a simple sequence of steps for solving it.

Task 4: Algorithms

Problem: An underground mine needs a system to monitor the condition of a ventilation fan.

- collect operating data,
- monitor vibration / temperature,
- compare measurements with normal conditions,
- **detect abnormal behavior**,
- generate a warning,
- provide information to the operator.

Algorithm

- read temperature,
- compare with acceptable range,
- determine normal / abnormal condition,
- send result to the monitoring system.

Task 5: Debugging / Troubleshooting

A ventilation fan monitoring system shows an abnormally high temperature, but the fan is actually operating normally.

- Suggest three possible causes of the problem and explain how you would check them.

Task 5: Debugging / Troubleshooting

A ventilation fan monitoring system shows an abnormally high temperature, but the fan is actually operating normally.

- faulty temperature sensor: compare its reading with another sensor or a reference measurement,
- incorrect sensor calibration: check calibration settings,
- data transmission error: compare the value measured by the sensor with the value received by the monitoring system,
- wrong threshold setting: check the configured acceptable temperature range,
- algorithm error: test the system using known input values.

Task 6: Integrated scenarion

Propose your own case and apply computational thinking to it.

Thanks!